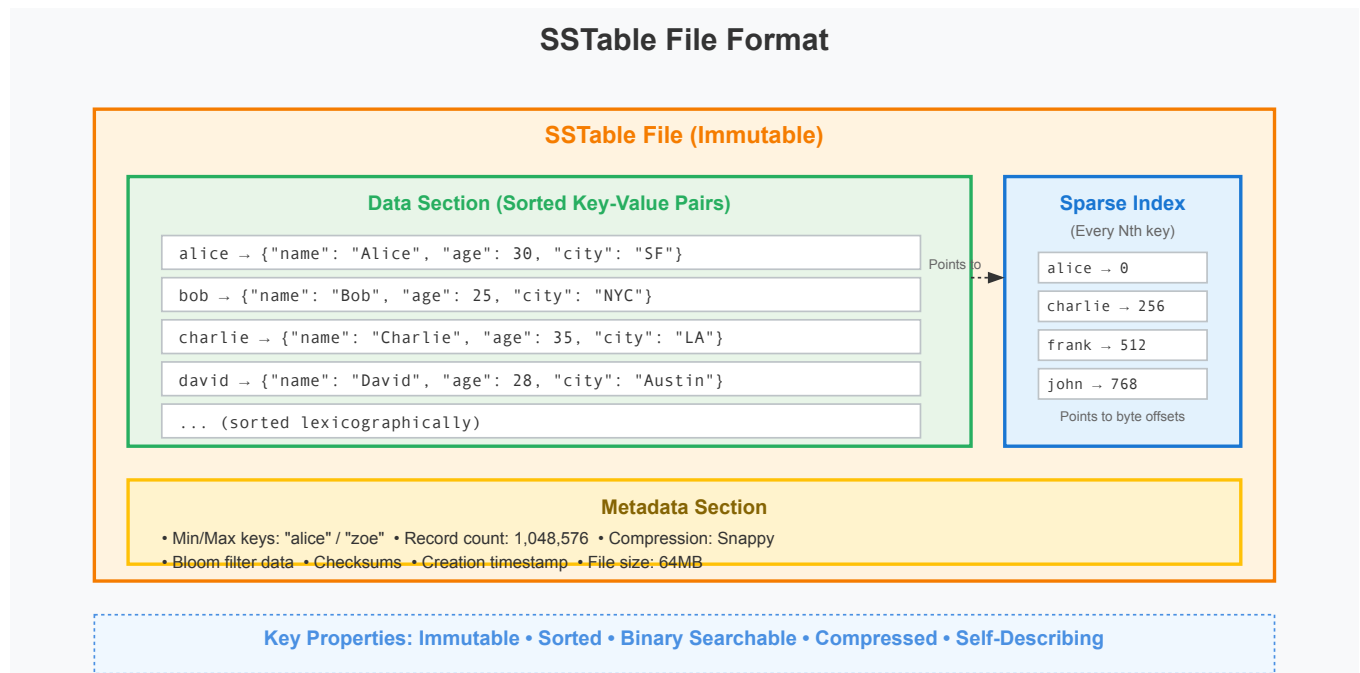


Sorted String Tables | System Design Primer



SStable

Sorted String Tables are **immutable file formats** that store key-value pairs in lexicographic order on disk. SSTables aren't complete database systems - they're the storage format that enables efficient binary search over large datasets. Modern distributed databases like Cassandra, RocksDB, and LevelDB use SSTables as their fundamental building blocks within larger storage engines.

The File Format Problem

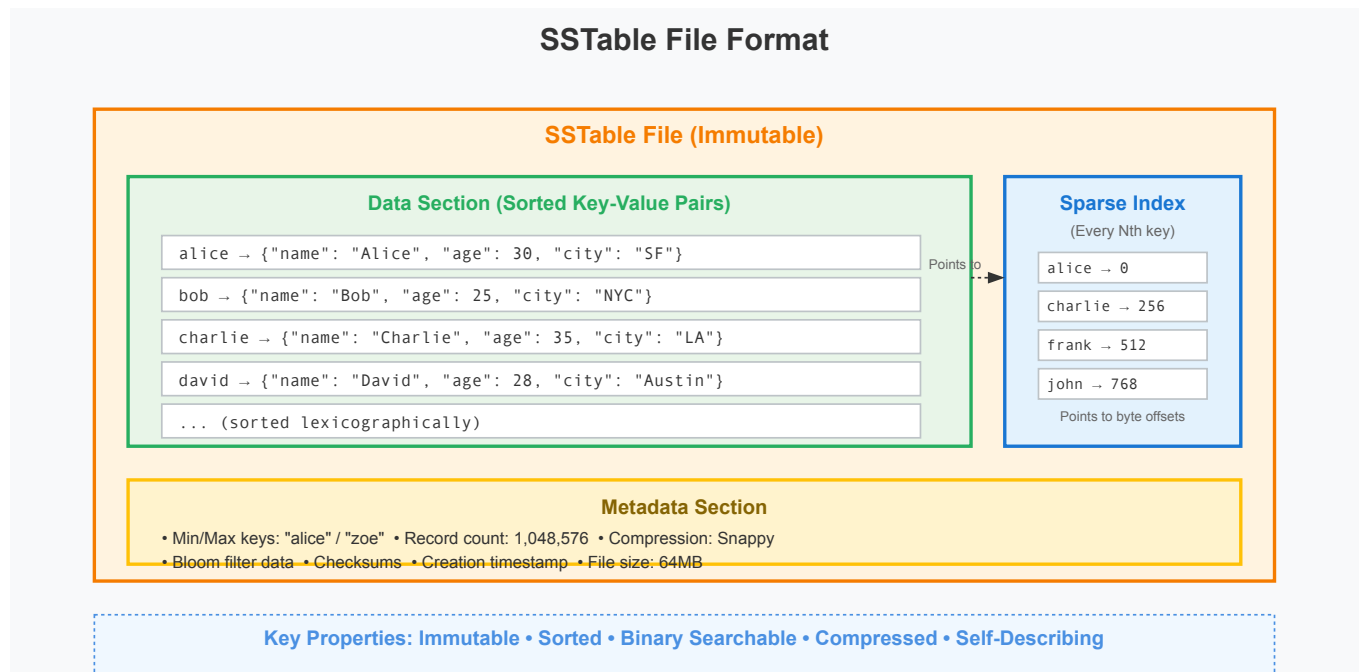
Database storage engines need efficient file formats for persisting sorted data to disk. Simple approaches like CSV files require sequential scanning, making lookups $O(n)$ for large datasets. Random access file formats can provide fast lookups but struggle with range queries and compression. SSTables solve this by combining sorted order with binary searchable indexes.

The idea behind SSTables came from Google's BigTable project: **store data in immutable, sorted files with sparse indexes**. Once written, an SStable never changes - updates and deletes are handled by creating new SSTables at the system level. This immutability enables aggressive caching, compression, and concurrent access without complex locking.

SSTables provide the foundation for [LSM trees](#), which handle the complete write path through memory buffers and multi-level compaction. Understanding SSTables as a file format helps explain why LSM-based systems achieve such high performance for both reads and writes.

SSTable File Format

An SSTable file contains three main components: the sorted data section, a sparse index, and optional metadata. The data section stores key-value pairs in lexicographic order, enabling binary search algorithms. The sparse index contains every Nth key and its file offset, trading memory usage for lookup performance. Metadata includes compression algorithms, bloom filter data, and file statistics.



The sparse index is crucial for performance. Instead of indexing every key (which would consume enormous memory), SSTables index every 16KB or 64KB block. This reduces memory overhead while still enabling fast seeks to the approximate location of any key.

Reading from an SSTable File Format

from typing import Optional, List, Tuple, Dict

"""Read-only SSTable implementation demonstrating the file format"""

```
def __init__(self, data_entries: List[Tuple[str, str]], block_size: int = 3):
```

```
# Simulate an already-written SSTable file
```

```
self.data = sorted(data_entries) # Data section (sorted)
```

```
self.block_size = block_size
```

```
# Build sparse index (every Nth entry)
```

```
self.sparse_index: List[Tuple[str, int]] = []
```

```
for i in range(0, len(self.data), block_size):
```

```
key, _ = self.data[i]
```

```
self.sparse_index.append((key, i))
```

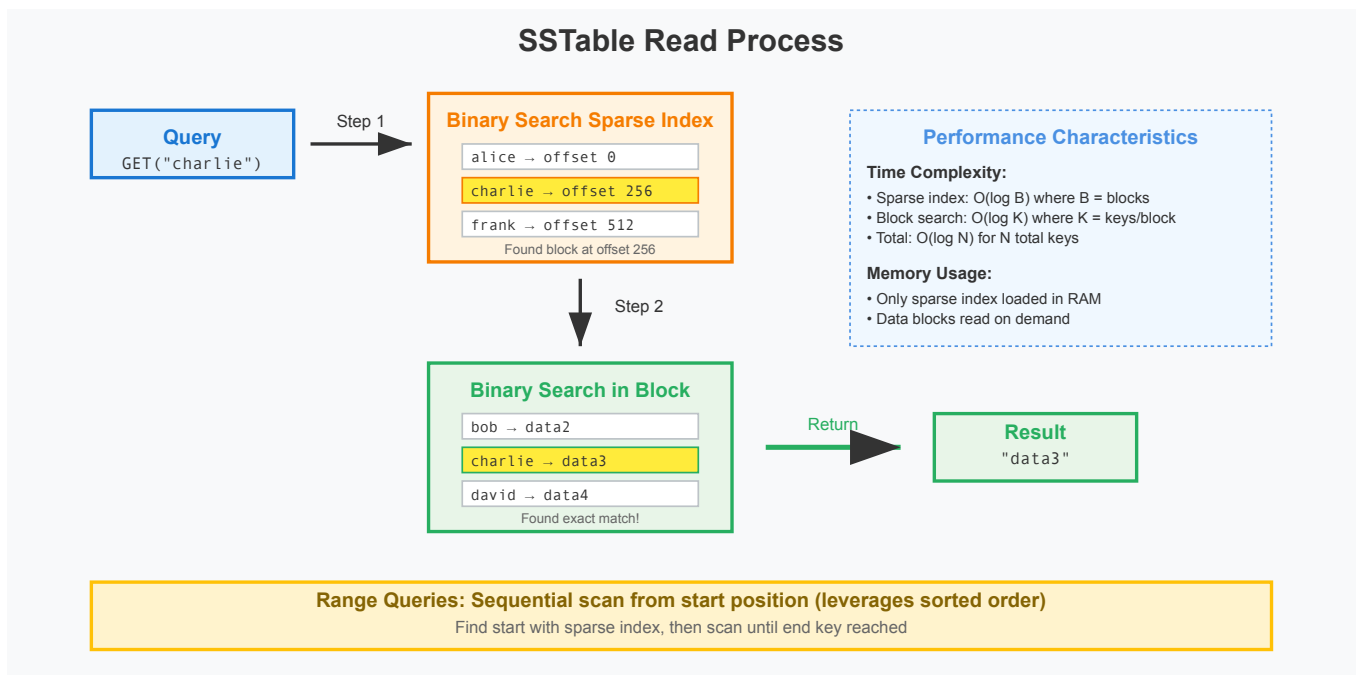
```
print(f"SSTable created with {len(self.data)} entries")
```

```
print(f"Sparse index built with {len(self.sparse_index)} entries (every {block_size} keys)")
```

```
def get(self, search_key: str) -> Optional[str]:
    """Binary search lookup using sparse index + linear scan"""
    print(f"\n--- Looking up '{search_key}' ---")
    if not self.data:
        # Step 1: Binary search in sparse index to find block
        index_keys = [key for key, _ in self.sparse_index]
        block_idx = bisect.bisect_right(index_keys, search_key) - 1
        block_idx = max(0, block_idx) # Handle edge case
        start_key, start_pos = self.sparse_index[block_idx]
        end_pos = len(self.data)
        if block_idx + 1 < len(self.sparse_index):
            _, end_pos = self.sparse_index[block_idx + 1]
```

Reading from SSTables

Reading from SSTables leverages the sorted structure and sparse index for efficient lookups. Point queries use a two-step process: binary search the sparse index to find the approximate block, then binary search within that block to find the exact key. This approach provides $O(\log n)$ performance while using minimal memory for the index.



Range queries are particularly efficient because of the sorted layout. Once you find the start position using the sparse index, you can sequentially scan until you reach the end of the range. This makes SSTables excellent for analytics workloads that need to process ranges of data.

Write Limitations and LSM Integration

SSTables are **immutable by design** - they provide no write operations after creation. Adding new keys or updating existing ones requires creating entirely new SStable files. This limitation is exactly why [LSM trees](#) exist: they provide the write path through memory buffers (memtables) and handle the complexity of managing multiple SStable files.

The immutable nature makes SSTables ideal building blocks for distributed systems. Files can be replicated, cached aggressively, and accessed concurrently without coordination. Multiple readers can access the same SStable simultaneously, and the files can be safely copied or moved without affecting ongoing operations.

When systems need to "update" data, they create new SSTables with newer values and rely on the read path to find the most recent version. Space reclamation and file management are handled by the storage engine's compaction processes, which merge multiple SSTables and delete obsolete files atomically.